



Departamento de Informática e Matemática Aplicada – DIMAp

Introdução a GraphQL

Prof. Nélio Cacho

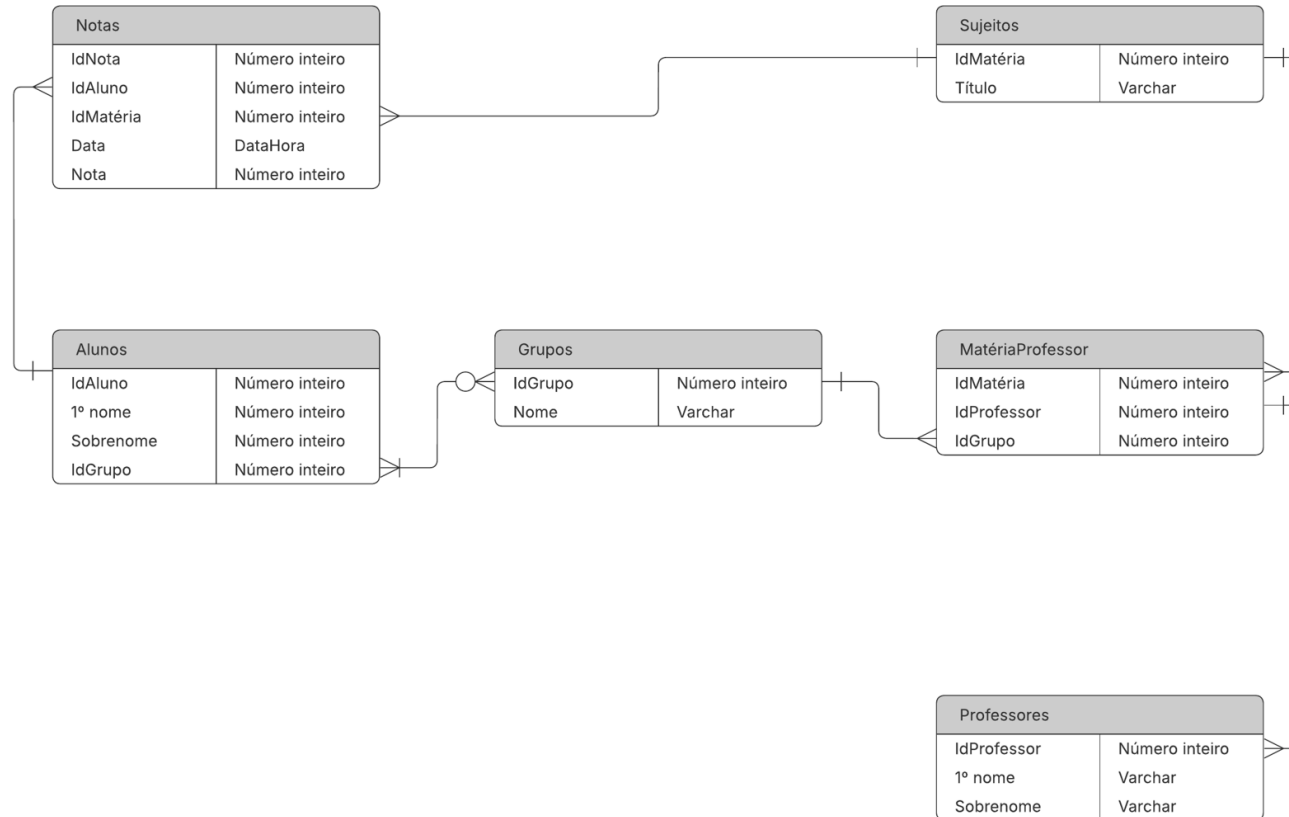
Universidade Federal do Rio Grande do Norte
neliocacho@dimap.ufrn.br

Hora de silenciar o celular...



- Manter o telefone celular sempre desligado/silencioso quando estiver em sala de aula;
- Nunca atender o celular na sala de aula.
- Não está autorizada a gravação ou reprodução desta aula

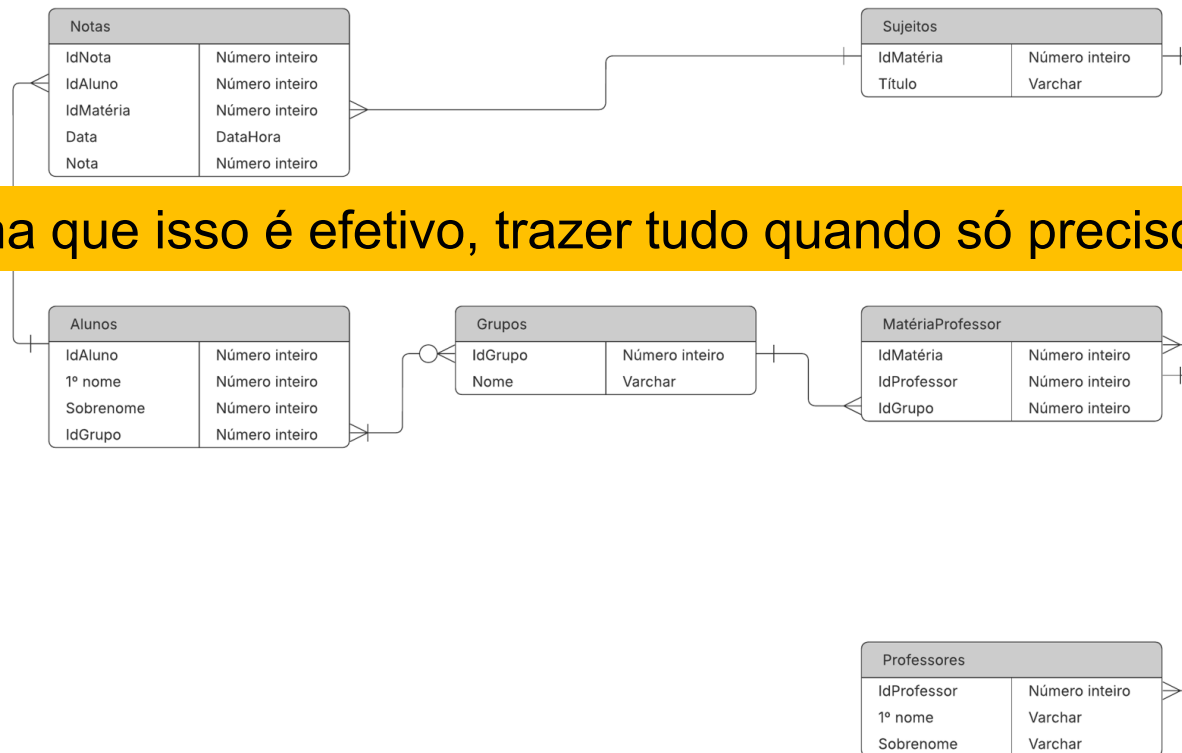
Considere esse cenário



- Desafio: Obter todas as notas do Aluno “Nélio” no Sujeito "Programação Distribuída"

Para essa tarefa, teríamos que fazer as seguintes consultas

1. `SELECT * FROM Alunos WHERE "1º nome" = 'Nélio';`
2. `SELECT * FROM Sujeitos WHERE Titulo = 'Programação Distribuída';`
3. `SELECT * FROM Notas WHERE IdAluno = :idAluno AND IdMatéria = :idMateria;`



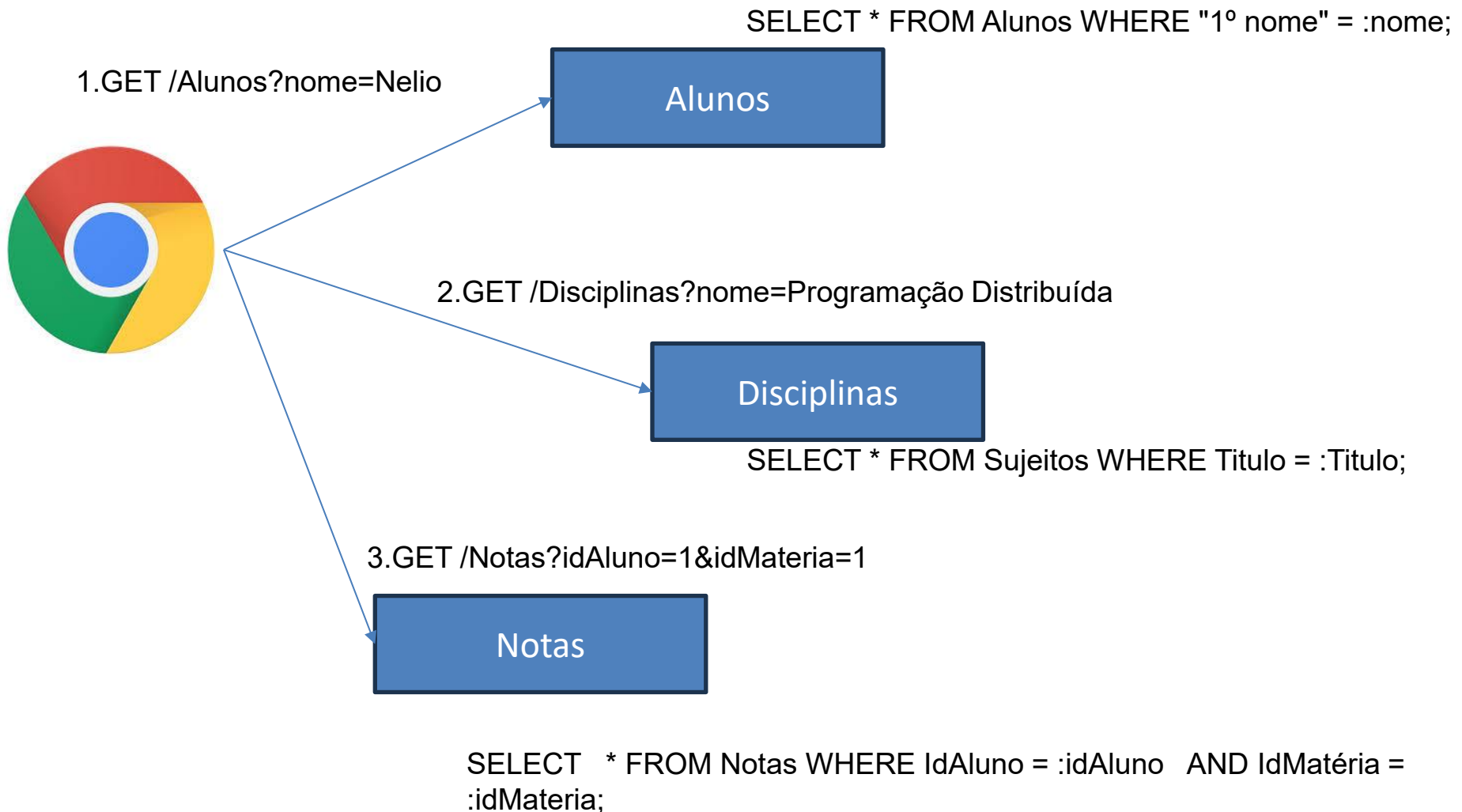
Você acha que isso é efetivo, trazer tudo quando só preciso do ID?

Para essa tarefa, teríamos que fazer as seguintes consultas

```
1.  SELECT n.Nota  FROM Notas n
      JOIN Alunos a ON n.IdAluno = a.IdAluno
      JOIN Sujeitos s ON n.IdMatéria = s.IdMatéria
      WHERE a."1º nome" = 'Nélio'
      AND s.Titulo = 'Programação Distribuída';
```

Essa seria a solução mais eficiente !!!

Considere agora isso feito como Serviços REST



Problemas do REST

- **Overfetching (dados em excesso):** O cliente recebe **mais dados do que precisa**. Isso acontece porque a API retorna um objeto fixo e completo, mesmo que só parte dele seja útil.
- **Underfetching (dados insuficientes):** O cliente **não recebe todos os dados necessários** em uma única requisição, sendo obrigado a fazer **várias chamadas** para compor a informação desejada.



Tratando Overfetching e Underfetching

- **Endpoint Proliferation (ou *API Endpoint Explosion*)**
 - É quando o número de endpoints cresce de forma descontrolada porque cada consumidor precisa de uma visão específica dos dados.
 - GET /orders
 - GET /orders/summary
 - GET /orders/with-items
 - GET /orders/with-items-and-payments
 - GET /orders/for-mobile
 - GET /orders/for-dashboard
 - GET /orders/for-admin
- Problemas gerados:
 - documentação cresce demais;
 - difícil descobrir qual endpoint usar;
 - manutenção cara;
 - aumenta chance de inconsistências;
 - versionamento vira um pesadelo.

Como resolver isso em Spring MVC

- Field Selection: Cliente escolhe quais atributos deseja
 - GET /users/1?fields=id,name,email
- Expansion Pattern: Cliente decide quais associações carregar.
 - GET /orders/123?expand=customer,items,payment
- Projections: Spring Data suporta projections.
- DTO Composition: Essa costuma ser a mais pragmática em projetos reais.
 - GET /users/1?include=orders,addresses

Como resolver isso em Spring MVC

Critério	REST + fields/include/projection	GraphQL
Overfetching	bom	excelente
Underfetching	bom	excelente
Endpoint explosion	bom	excelente
Curva de aprendizado	baixa	alta
Performance previsível	excelente	média
Caching	excelente	fraco
Observabilidade	excelente	média
Flexibilidade	média	excelente
Evolução da API	média	excelente
Segurança operacional	simples	complexa

GraphQL: Uma nova forma de consultar APIs

- Linguagem de consulta de dados para APIs.
- Criado pelo Facebook em 2012 e liberado em 2015.
- Permite que o cliente especifique exatamente quais dados precisa.
- GraphQL é fortemente tipada.
- GraphQL elimina os problema de Overfetching e Underfetching

GraphQL: Uma nova forma de consultar APIs



```
SELECT n.Nota FROM Notas n
JOIN Alunos a ON n.IdAluno = a.IdAluno
JOIN Sujeitos s ON n.IdMatéria = s.IdMatéria
WHERE a."1º nome" = 'Nélcio'
AND s.Título = 'Programação Distribuída';
```

GraphQL

Alunos

Disciplinas

Notas

GraphQL é fortemente Tipada

- O Schema define os tipos de dados disponíveis e seus relacionamentos.
- São como Schemas de bancos de dados.
- A partir dos schemas, os clientes sabem exatamente o que podem consultar.
- São geralmente definidos no arquivo *schema.graphqls*

```
type Aluno{  
  idAluno: ID  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
}  
  
type Sujeitos {  
  idMateria: ID  
  titulo: String  
}  
  
type Notas{  
  idAluno: ID  
  idMateria: ID  
  valorNota: Float  
}
```

Tipos básicos do GraphQL

- São os tipos básicos.
 - Int: número inteiro.
 - Float: número decimal.
 - String: texto.
 - Boolean: verdadeiro/falso.
 - ID → identificador único (string ou número).

Coleções em GraphQL

- Para representar Coleções, usamos:
 - [...] exemplo [Int] que seria equivalente a List<Integer>

```
type Aluno{  
  idAluno: ID  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  notas: [Notas]  
}
```

Campos obrigatórios GraphQL

- No GraphQL, o ! na definição de um **schema** significa “**não nulo**” (ou **non-nullable**). Ou seja, o campo **deve sempre ter um valor**, e não pode retornar null.

```
type Aluno{
  idAluno: ID!
  primeiroNome: String
  sobrenome: String
  idGrupo: Int
  notas: [Notas]
}

type Sujeitos {
  idMateria: ID!
  titulo: String
}

type Notas{
  idAluno: ID
  idMateria: ID
  valorNota: Float!
}
```


Enumeração no GraphQL

- No GraphQL, um **Enum** (enumeration) é um tipo especial que define **um conjunto de valores possíveis para um campo**. Ele serve para garantir que o campo só possa assumir valores pré-definidos, como “status” ou “categoria”.

```
type Aluno{  
  idAluno: ID!  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  notas: [Notas]  
  status: StatusAluno  
}
```

```
enum StatusAluno{  
  MATRICULADO  
  TRANCADO  
}
```

Root types no GraphQL

- **Root types** são os tipos que definem **como o cliente interage com o servidor**:
 - **Query**: Obter dados do servidor (como “ler” informações). Similar ao GET no REST
 - **Mutation**: Modificar dados no servidor (criar, atualizar, deletar). Similar ao POST, PUT, DELETE do REST
 - **Subscription**: Receber dados em **tempo real** do servidor. Funciona via WebSocket ou protocolo compatível, permitindo “push” de eventos.

Vamos criar um projeto em Spring...



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin

☒ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT) ☒ 4.0.6

☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☒ 21 ☐ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring for GraphQL WEB

Build GraphQL applications with Spring for GraphQL and GraphQL Java.

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Dependências importantes

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>
</dependency>
```

Definindo o GraphQL Schema

```
src > main > resources > graphql >  schema.graphqls
```

```
1  type Query{  
2    getNomeAluno: String  
3  }
```

Definindo o Controller

- No *Spring for GraphQL*, a anotação `@QueryMapping` serve para **mapear um método Java a um campo do tipo Query no schema GraphQL**.
- O nome do método (ou o valor no `@QueryMapping("...")`) precisa bater com o campo definido no schema.

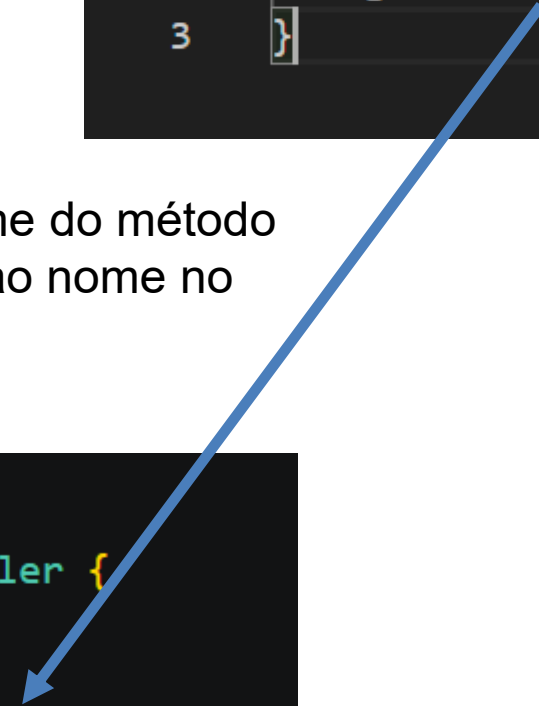
Definindo o Schema e Controller

```
src > main > resources > graphql > schema.graphqls
1  type Query{
2      getNomeAluno: String
3  }
```

Note que o nome do método
deve ser igual ao nome no
Schema

```
@Controller
public class SigaaController {

    @QueryMapping
    public String getNomeAluno() {
        return "Nélio Cacho";
    }
}
```

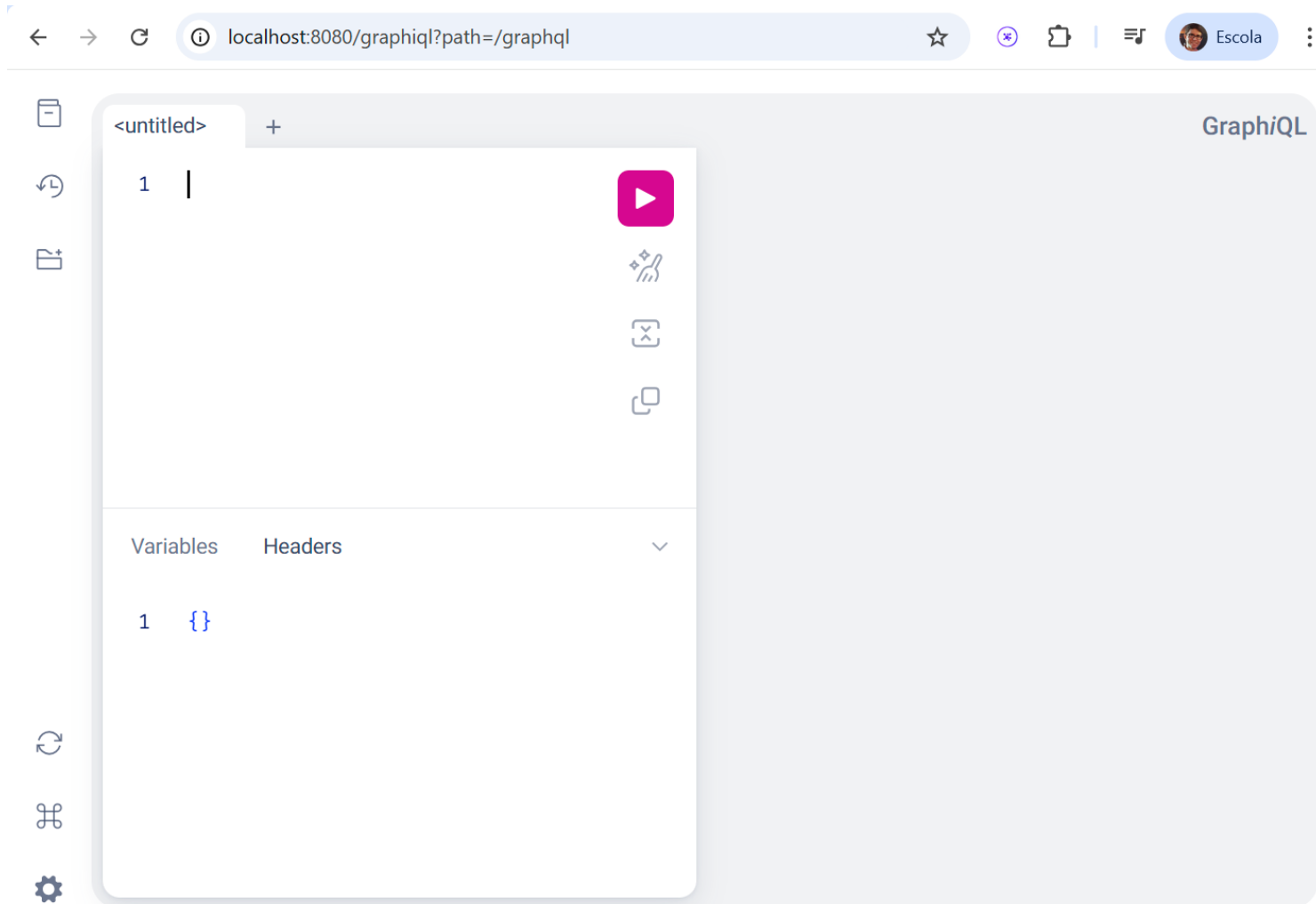


GraphQL Interactive IDE

- GraphiQL é a sigla para "GraphQL Interactive IDE" (Ambiente de Desenvolvimento Integrado Interativo). Ele é uma ferramenta visual que facilita a interação com APIs GraphQL diretamente no navegador. Pense nele como o Postman ou o Insomnia, mas projetado especificamente para a linguagem GraphQL.

```
src > main > resources > ≡ application.properties
1   spring.application.name=projetographql
2
3   spring.graphql.graphiql.enabled=true
```


GraphQL Interactive IDE



<http://localhost:8080/graphql?path=/graphql>

GraphQL Interactive IDE

The screenshot displays the GraphQL Interactive IDE interface in a web browser. The browser's address bar shows the URL `localhost:8080/graphql?path=/graphql`. The interface is divided into several sections:

- Explorer:** Located on the left, it shows a list of queries. The first query is named `MyQuery` and contains the operation `getNomeAluno`. Below the list, there is an "Add new" button and a dropdown menu set to "Query".
- Query Editor:** The central area where the GraphQL query is written. It is titled "MyQuery" and contains the following query:

```
1 query MyQuery {  
2   |   getNomeAluno  
3 }
```
- Variables:** A section below the query editor for defining variables. It is currently empty, showing only the opening curly braces:

```
1 {}  
2
```
- Response:** On the right side, the JSON response of the query is displayed:

```
{  
  "data": {  
    "getNomeAluno": "Nélio Cacho"  
  }  
}
```

On the far left, there is a vertical sidebar with icons for file management, undo/redo, and settings. On the far right, there is a user profile icon labeled "Escola".

Quando não existe coincidência...

```
@Controller
public class SigaaController {

    @QueryMapping
    public String getNomeAlunoNovo() {
        return "Nélio Cacho";
    }
}
```

MyQuery

MyQuery x

+

```
1 query MyQuery {
2   |   getNomeAluno
3 }
```



```
{
  | "data": {
  |   "getNomeAluno": null
  | }
}
```

Ajuste o QueryMapping com um parâmetro

```
@Controller
public class SigaaController {

    @QueryMapping(name = "getNomeAluno")
    public String getNomeAlunoNovo() {
        return "Nélio Cacho";
    }
}
```

MyQuery

+

```
1 query MyQuery {
2   |   getNomeAluno
3 }
```



```
{
  "data": {
    "getNomeAluno": "Nélio Cacho"
  }
}
```

Passando parâmetros para o QueryMapping

- No *Spring for GraphQL*, você passa parâmetros para o **@QueryMapping** usando a anotação **@Argument**. Isso mapeia diretamente os **argumentos do campo GraphQL** para os **parâmetros do método Java**.
- Se colocar ! no schema, indica que o parâmetro não pode ser nulo.
- O nome dos parâmetros precisam coincidir com a implementação

```
type Query{  
  getNomeAluno: String  
  getNomeAlunoById(id: Int! ): String  
}
```

```
@QueryMapping  
public String getNomeAlunoById(@Argument int id) {  
  return "Nélio Cacho " + id;  
}
```

Passando parâmetros para o QueryMapping

MyQuery

+

```
1 query MyQuery {  
2   |  getNomeAlunoById(id: 10)  
3 }
```



GraphQL

```
{  
  "data": {  
    "getNomeAlunoById": "Nélío Cacho 10"  
  }  
}
```

Retornando um Tipo Customizável

- Até o momento, a maioria das nossas queries retorna tipos simples (String, Int, Boolean). Mas e se a resposta for um objeto complexo, como um Aluno ou um Produto?
- Para tanto, deve-se primeiro criar um tipo no schema, como já foi feito.
- Depois associar um retorno a esse tipo:

```
type Aluno{  
  idAluno: ID!  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  status: StatusAluno  
}
```

```
type Query{  
  alunoById(id: Int! ): Aluno  
  todosalunos:[Aluno]  
}
```

Retornando um Tipo Customizável

- Feito isso, vamos criar o Model, Service e Controller no nosso backend:

```
public record Aluno(  
    String idAluno,  
    String primeiroNome,  
    String sobrenome,  
    Integer idGrupo,  
    StatusAluno status  
) {}
```

```
public enum StatusAluno {  
    ATIVO,  
    ... INATIVO,  
    ... TRANCADO,  
    ... CONCLUIDO  
}
```


Retornando um Tipo Customizável

- Nosso Service:

```
@Service
public class AlunoService {
    private final Map<String, Aluno> alunos = new HashMap<>();
    public AlunoService() {
        alunos.put(key: "1", new Aluno(idAluno: "1", primeiroNome: "João", sobrenome: "Silva", idGrupo: 1, StatusAluno.ATIVO));
        alunos.put(key: "2", new Aluno(idAluno: "2", primeiroNome: "Maria", sobrenome: "Souza", idGrupo: 2, StatusAluno.ATIVO));
        alunos.put(key: "3", new Aluno(idAluno: "3", primeiroNome: "Thiago", sobrenome: "Luiz", idGrupo: 3, StatusAluno.ATIVO));
        alunos.put(key: "4", new Aluno(idAluno: "4", primeiroNome: "Josefa", sobrenome: "Barbosa", idGrupo: 4, StatusAluno.INATIVO));
        alunos.put(key: "5", new Aluno(idAluno: "5", primeiroNome: "Luiz", sobrenome: "Gonzaga", idGrupo: 5, StatusAluno.ATIVO));
    }
    public Aluno findById(String id) {
        return alunos.get(id);
    }
    public List<Aluno> findAll() {
        return new ArrayList<>(alunos.values());
    }
}
```

Retornando um Tipo Customizável

- Nosso Controller:

```
@Controller
public class AlunoController {

    private final AlunoService alunoService;

    ← AlunoService
    public AlunoController(AlunoService alunoService) {
        this.alunoService = alunoService;
    }

    @QueryMapping(name = "alunoById")
    public Aluno getAlunoById(@Argument String id) {
        return alunoService.findById(id);
    }

    @QueryMapping(name = "todosalunos")
    public List<Aluno> getAlunos() {
        return alunoService.findAll();
    }
}
```

Vamos testar

- Posso obter todos os campos do retorno:

The image shows a GraphQL IDE interface with two main panels. The left panel, titled 'MyQuery', contains a query definition. The right panel, titled 'GraphQL', shows the JSON response for that query. Below the query editor, there are tabs for 'Variables' and 'Headers', with 'Variables' currently selected, showing an empty object `{}`.

```
1 query MyQuery {  
2   todosalunos {  
3     primeiroNome  
4     status  
5     idAluno  
6     idGrupo  
7     sobrenome  
8   }  
9 }
```

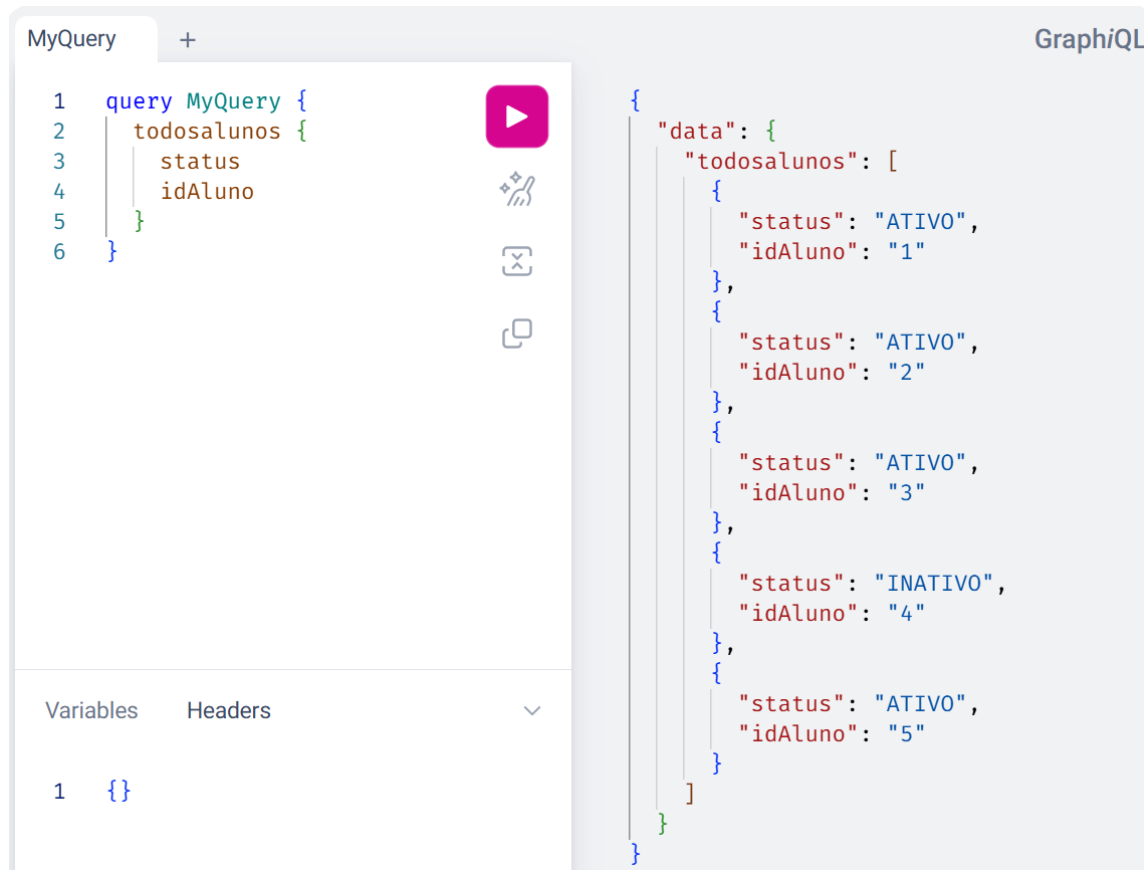
```
{  
  "data": {  
    "todosalunos": [  
      {  
        "primeiroNome": "João",  
        "status": "ATIVO",  
        "idAluno": "1",  
        "idGrupo": 1,  
        "sobrenome": "Silva"  
      },  
      {  
        "primeiroNome": "Maria",  
        "status": "ATIVO",  
        "idAluno": "2",  
        "idGrupo": 2,  
        "sobrenome": "Souza"  
      },  
      {  
        "primeiroNome": "Thiago",  
        "status": "ATIVO",  
        "idAluno": "3",  
        "idGrupo": 2,  
        "sobrenome": "Luiz"  
      },  
      {  
        "primeiroNome": "Josefa",  
        "status": "INATIVO",  
        "idAluno": "4",  
        "idGrupo": 2,  
        "sobrenome": "Barbosa"  
      },  
      {  
        "primeiroNome": "Luiz",  
        "status": "ATIVO",  
        "idAluno": "5",  
        "idGrupo": 2,  
        "sobrenome": "Gonzaga"  
      }  
    ]  
  }  
}
```

Variables Headers

1 {}

Vamos testar

- Ou posso, obter apenas alguns campos selecionados para reduzir o payload do retorno.



The screenshot displays the GraphQL IDE interface. On the left, a query named 'MyQuery' is defined with the following structure:

```
1 query MyQuery {  
2   todosalunos {  
3     status  
4     idAluno  
5   }  
6 }
```

Below the query editor, the 'Variables' section shows an empty object: `{}`.

On the right, the JSON response is shown, indicating a successful query execution:

```
{  
  "data": {  
    "todosalunos": [  
      {  
        "status": "ATIVO",  
        "idAluno": "1"  
      },  
      {  
        "status": "ATIVO",  
        "idAluno": "2"  
      },  
      {  
        "status": "ATIVO",  
        "idAluno": "3"  
      },  
      {  
        "status": "INATIVO",  
        "idAluno": "4"  
      },  
      {  
        "status": "ATIVO",  
        "idAluno": "5"  
      }  
    ]  
  }  
}
```

Tipos “Input”

- Para receber tipos como parâmetros de entrada, deve-se criar tipos “input”.
- Criar um tipo de **input** é uma prática recomendada no GraphQL!
- **Por que usar input?**
 - **É uma recomendação da especificação que busca:**
 - **Organização:** Em vez de ter uma longa lista de argumentos em sua query (por exemplo, todososAlunosPorFaixa(idmax: ID, idMin:Int, ...)), você agrupa tudo em um único objeto de entrada.
 - **Reutilização:** Você pode reutilizar o mesmo tipo de input em diferentes mutações (por exemplo, para criar e atualizar um Aluno).
 - **Evolução da API:** Se você precisar adicionar um novo campo no futuro, basta adicioná-lo ao input sem alterar a assinatura da mutação.

Tipos Input

- **input FiltroAlunoPorID { ... }**: A palavra-chave **input** é usada para declarar um tipo de objeto que pode ser passado como argumento.
- A sintaxe é a mesma de um *type* normal.

```
input FiltroAlunoPorID {  
  minId: Int!  
  maxId: Int!  
}
```

Definindo uma Query com Input

- Pode-se definir uma query com input.

```
type Query{  
  todososAlunosPorFaixaId(filtro: FiltroAlunoPorID): [Aluno]  
}  
  
input FiltroAlunoPorID {  
  minId: Int!  
  maxId: Int!  
}
```

Definindo uma Query com Input

- Definindo o Controller e Service

```
@QueryMapping(name = "todososAlunosPorFaixaId")
public List<Aluno> getAlunosPorFaixaID(@Argument FiltroAlunoPorID filtro) {
    return alunoService.findAlunoPorFiltroIDGrupo(filtro);
}
```

```
public List<Aluno> findAlunoPorFiltroIDGrupo(FiltroAlunoPorID filtro) {
    return alunos.values()
        .stream()
        .filter(aluno ->
            aluno.idGrupo() >= filtro.minId() &&
            aluno.idGrupo() <= filtro.maxId())
        .toList();
}
```


Definindo uma Query com Input

MyQuery2

+

GraphiQL

```
1 query MyQuery2 {  
2   todososAlunosPorFaixaId(filtro: {maxId: 10, minId: 1}) {  
3     idAluno  
4     idGrupo  
5   }  
6 }
```



Variables

Headers

```
1 {}
```

```
{  
  "data": {  
    "todososAlunosPorFaixaId": [  
      {  
        "idAluno": "1",  
        "idGrupo": 1  
      },  
      {  
        "idAluno": "2",  
        "idGrupo": 2  
      },  
      {  
        "idAluno": "3",  
        "idGrupo": 3  
      },  
      {  
        "idAluno": "4",  
        "idGrupo": 4  
      },  
      {  
        "idAluno": "5",  
        "idGrupo": 5  
      }  
    ]  
  }  
}
```

Qual a diferença de REST para GraphQL mesmo?

- Suponha que tenha esse Schema e seu respectivo Controller:

```
type Query{  
  getNomeAluno: String  
  getNomeAlunoById(id: Int! ): String  
  getDisciplina(id: Int): String  
}
```

```
@Controller  
public class SigaaController {  
  
  @QueryMapping(name = "getNomeAluno")  
  public String getNomeAlunoNovo() {  
    return "Nélio Cacho";  
  }  
  
  @QueryMapping  
  public String getNomeAlunoById(@Argument int id) {  
    return "Nélio Cacho " + id;  
  }  
  
  @QueryMapping  
  public String getDisciplina(@Argument int id) {  
    return "Programação Distribuida - DIM000 " + id;  
  }  
}
```

Qual a diferença de RESP para GraphQL mesmo?

- Enquanto no GraphQL, com apenas uma requisição se obtém todos os dados, no REST, seria precisa fazer 3 requisições

MyQuery

+

```
1 query MyQuery {  
2   |  getNomeAlunoById(id: 50)  
3   |  getNomeAluno  
4   |  getDisciplina(id: 10)  
5 }
```



GraphiQL

```
{  
  "data": {  
    "getNomeAlunoById": "Nélio Cacho 50",  
    "getNomeAluno": "Nélio Cacho",  
    "getDisciplina": "Programação  
Distribuida - DIM000 10"  
  }  
}
```

Como GraphQL funciona?

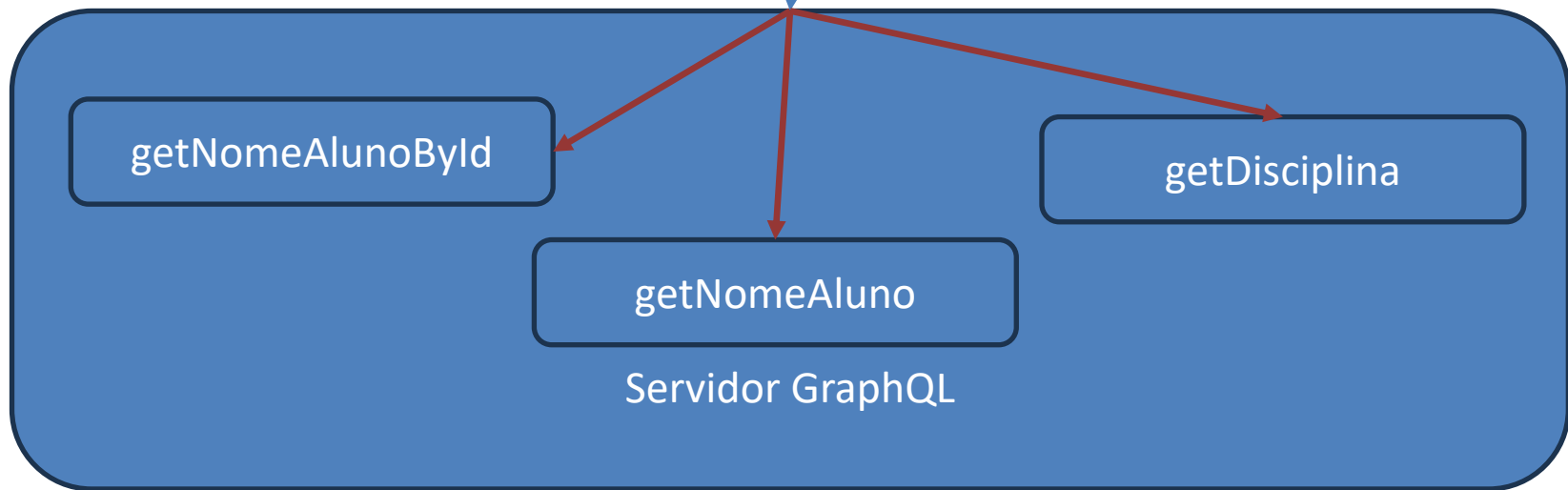
- O GraphQL em si não define um protocolo de transporte.
- Ele é apenas uma **linguagem de consulta** e um **runtime de execução**.
- Ou seja, você pode transportar uma requisição GraphQL de várias formas diferentes.
 - **Queries e Mutations** normalmente são enviadas como HTTP **POST** para um endpoint único, geralmente /graphql. O corpo da requisição é um **JSON** com pelo menos a chave query.
 - **Subscriptions** são geralmente implementados via WebSocket

Como GraphQL opera?



Como GraphQL opera?

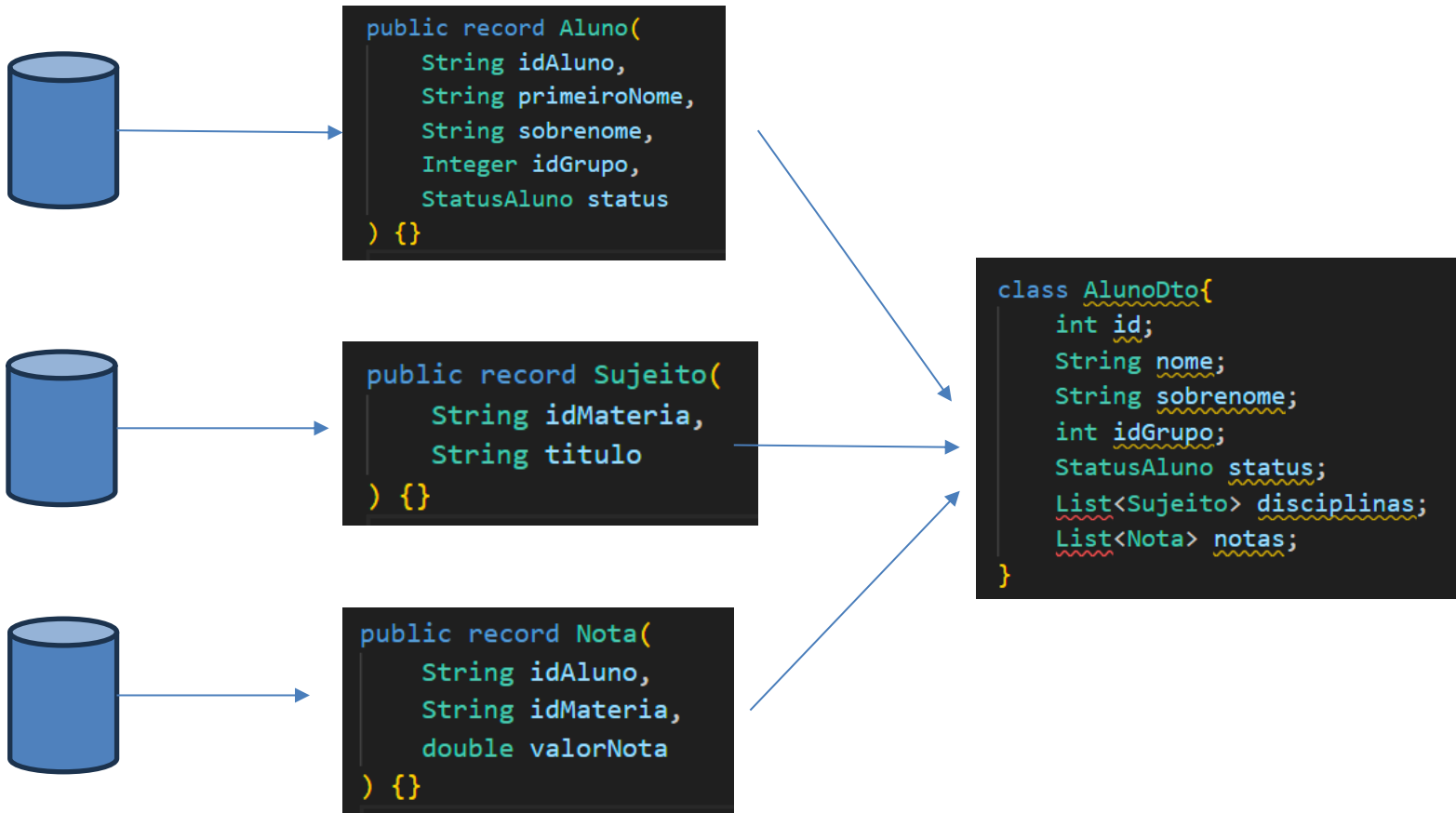
```
query MyQuery {  
  getNomeAlunoById(id: 50)  
  getNomeAluno  
  getDisciplina(id: 10)  
}
```



Quando uma consulta é realizada, os métodos no Controller são invocados de forma paralela, e ao final, um ZIP é chamado para concatenar os resultados.

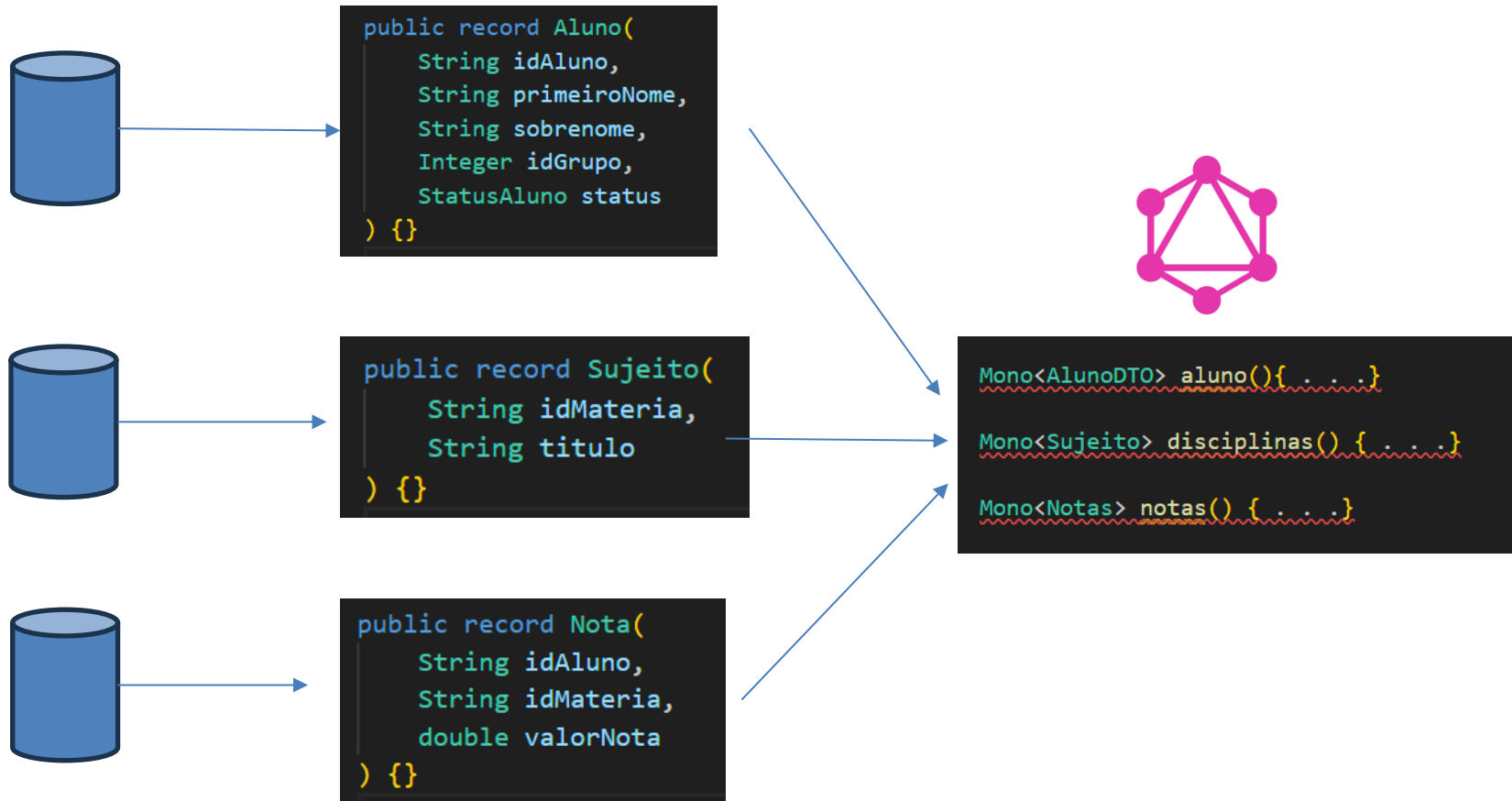
Campos Aninhados em REST

- Utilizando REST, para compor campos aninhados, é preciso fazer algo como descrito abaixo, onde uma API retorna um AlunoDTO compondo vários serviços.



Campos Aninhados em GraphQL

- Utilizando GraphQL, deve-se fornecer rotas separadas e o GraphQL faz a agregação.



Mapeando Campos Aninhados

- Note que o `@QueryMapping` é um alias para `@SchemaMapping` do tipo “Query”, porque no arquivo `Schema.graphqls`, a rota “`todosalunos`” é filho de `Query`(tipo root).

```
type Query{  
  todosalunos:[Aluno]  
}
```

```
@QueryMapping(name = "todosalunos")
```

yMapp... \file:\C:\Users\nelio\.m2\repository\org\springframework\graphql\spring-graphql\1.4.1\spring-graphql-1.4.1-sources.jar!org\... - Definiti... X

```
* @author Rossen Stoyanchev  
* @since 1.0.0  
*/  
@Target(ElementType.METHOD)  
@Retention(RetentionPolicy.RUNTIME)  
@Documented  
@SchemaMapping(typeName = "Query")  
public @interface QueryMapping {
```

QueryMapping.java \file:\C:\User... 1

interface QueryMapping {

> QueryMapping.class \spring-grap... 1

Mapeando Campos Aninhados

- Para mapear campos aninhados no GraphQL, é preciso usar a anotação `@SchemaMapping`.
- Esta anotação faz o mapeamento de campos específicos do seu schema GraphQL em controladores.
- Em essência, ele permite que você resolva "campos aninhados" (nested fields) que não são atributos diretos de uma classe Java.
- Pense nele como uma forma de construir o grafo de dados, resolvendo partes da resposta que exigem uma lógica de negócio adicional.

Mapeando Campos Aninhados

- Portanto, para o schema descrito abaixo, preciso definir um controller cujo *type* é Aluno, já que aluno é o tipo “pai” do campo “disciplinas”.

```
type Query{  
  todosalunos:[Aluno]  
}  
  
type Aluno{  
  idAluno: ID!  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  status: StatusAluno  
  disciplinas(limit: Int!): [Sujeitos]  
}
```

Esse parâmetro
representa o tipo “Pai”

```
@SchemaMapping(typeName = "Aluno")  
public List<AlunoSujeitos> disciplinas(Aluno aluno){  
  System.out.println("Metodo do aluno chamado " + aluno.primeiroNome());  
  return this.sujeitoAlunoService.disciplinasporAluno(aluno.idAluno());  
}
```

Mapeando Campos Aninhados

- Assim ficaria o Service para obter as disciplinas do aluno.

```
@Service
public class SujeitoAlunoService {

    private final Map<String, List<AlunoSujeitos>> sujeitos = Map.of(
        k1: "1", List.of(new AlunoSujeitos(idMateria: 1, titulo: "Programação Distribuída"), new AlunoSujeitos(idMateria: 2, titulo: "História")),
        k2: "2", List.of(new AlunoSujeitos(idMateria: 3, titulo: "Programação Concorrente"), new AlunoSujeitos(idMateria: 4, titulo: "Química")),
        k3: "3", List.of(new AlunoSujeitos(idMateria: 5, titulo: "Física"), new AlunoSujeitos(idMateria: 6, titulo: "Geografia"))
    );

    public List<AlunoSujeitos> disciplinasporAluno(String idaluno) {
        return sujeitos.getOrDefault(idaluno, Collections.emptyList());
    }
}
```

Resultado

MyQuery2

+

```
1 query MyQuery2 {  
2   todosalunos {  
3     disciplinas(limit: 10) {  
4       idMateria  
5       titulo  
6     }  
7     primeiroNome  
8   }  
9 }
```



Variables

Headers

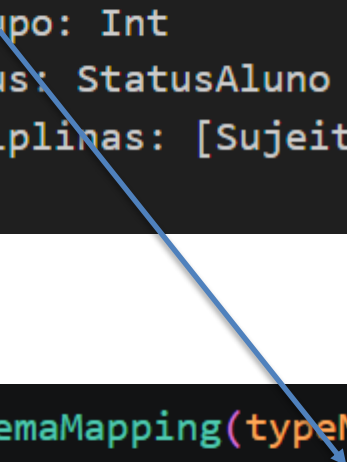


```
1 {}
```

```
{  
  "data": {  
    "todosalunos": [  
      {  
        "disciplinas": [  
          {  
            "idMateria": "1",  
            "titulo": "Programação Distribuída"  
          },  
          {  
            "idMateria": "2",  
            "titulo": "História"  
          }  
        ],  
        "primeiroNome": "João"  
      },  
      {  
        "disciplinas": [  
          {  
            "idMateria": "3",  
            "titulo": "Programação Concorrente"  
          },  
          {  
            "idMateria": "4",  
            "titulo": "Química"  
          }  
        ],  
        "primeiroNome": "Maria"  
      }  
    ]  
  }  
}
```

Pode-se substituir o valor de um campo por um controller

```
type Query{  
  todosalunos:[Aluno]  
}  
type Aluno{  
  idAluno: ID!  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  status: StatusAluno  
  disciplinas: [Sujeitos]  
}
```



```
@SchemaMapping(typeName = "Aluno")  
public Integer idGrupo(Aluno aluno){  
  System.out.println("Metodo do aluno chamado " + aluno.primeiroNome());  
  return 1000;  
}
```

Pode-se substituir o valor de um campo por um controller

MyQuery +

```
1 query MyQuery {
2   todosalunos {
3     disciplinas {
4       titulo
5     }
6     idAluno
7     primeiroNome
8     idGrupo
9   }
}
```

Variables Headers

1 {}

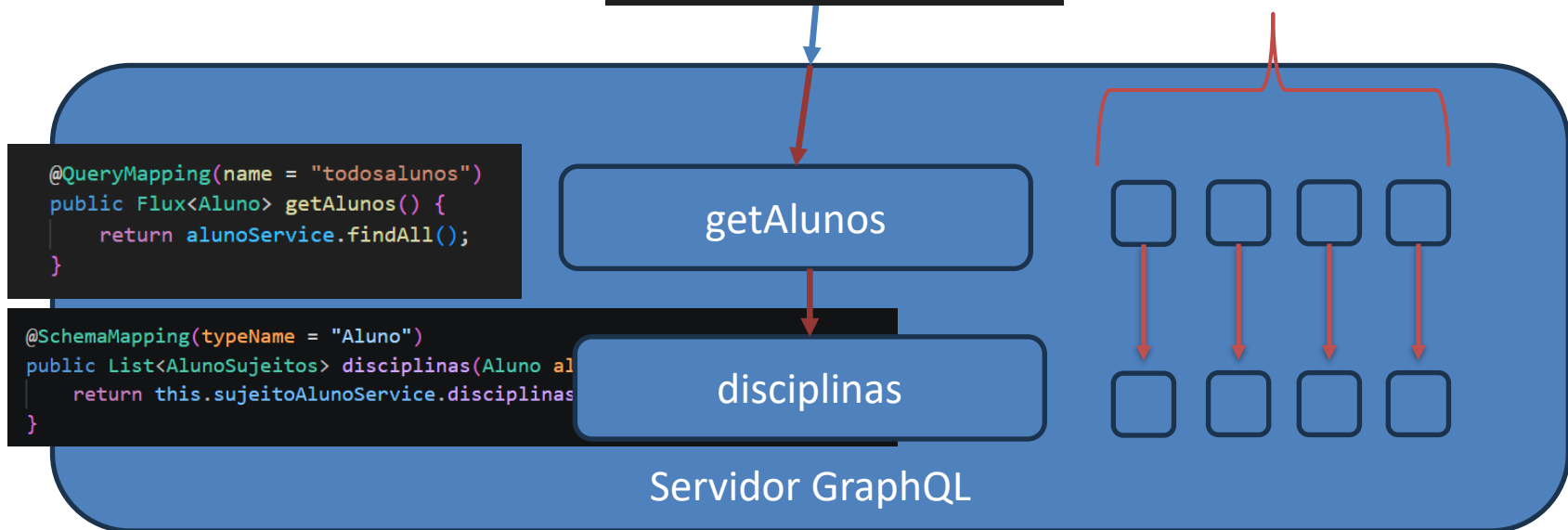
GraphiQL

```
{
  "data": {
    "todosalunos": [
      {
        "disciplinas": [
          {
            "titulo": "Programação
            Distribuída"
          },
          {
            "titulo": "História"
          }
        ],
        "idAluno": "1",
        "primeiroNome": "João",
        "idGrupo": 1000
      },
      {
        "disciplinas": [
          {
            "titulo": "Programação
            Concorrente"
          },
          {
            "titulo": "Química"
          }
        ],
        "idAluno": "2",
        "primeiroNome": "Maria",
        "idGrupo": 1000
      }
    ]
  }
}
```

Como GraphQL opera?

```
type Query{  
  todosalunos:[Aluno]  
}  
  
type Aluno{  
  idAluno: ID!  
  primeiroNome: String  
  sobrenome: String  
  idGrupo: Int  
  status: StatusAluno  
  disciplinas(limit: Int!): [Sujeitos]  
}
```

Disciplina é chamado várias vezes para compor a lista, isso se trata do “**N + 1 Problem**”



O Problema N+1: O que é?

- O problema N+1 ocorre quando um sistema faz:
 - 1 consulta para buscar a lista de todos os alunos.
 - + N consultas adicionais (onde N é o número de alunos) para buscar as disciplinas de cada aluno, uma por uma.
- No nosso exemplo, como temos 5 alunos cadastrados, seriam 6 consultas (1 para os alunos + 5 para as disciplinas), o que prejudica a performance.

Soluções para o N+1 no GraphQL

- **Batching (em GraphQL):** agrupar as solicitações de dados relacionadas.
- O **Batching** coleta todas as requisições de um mesmo tipo (ex: buscar várias disciplinas por ID) e as executa em uma única consulta otimizada.
- Ele resolve o problema de forma automática, garantindo que o GraphQL seja eficiente.

Soluções para o N+1 no GraphQL

- No service, ficou assim:

```
public List<AlunoSujeitos> disciplinasporAlunoS(List<String> idalunos) {  
    return idalunos.stream()  
        .map(id->sujeitos.getDefault(id, Collections.emptyList()))  
        .flatMap(List::stream)  
        .collect(Collectors.toList());  
}
```

- No controller:

```
@BatchMapping(typeName = "Aluno")  
public Map<Aluno, List<AlunoSujeitos>> disciplinas(List<Aluno> alunos){  
    System.out.println(x: "Esse método foi chamado apenas uma vez");  
    return alunos.stream()  
        .collect(Collectors.toMap(  
            aluno -> aluno,  
            aluno -> sujeitoAlunoService  
                .disciplinasporAluno(aluno.idAluno())  
        ));  
}
```

Soluções para o N+1 no GraphQL

- **@BatchMapping(typeName = "Aluno")**: Ela instrui o Spring for GraphQL a chamar este método para resolver o campo "disciplinas" no tipo Aluno do seu schema. A grande diferença do @SchemaMapping é que ele não é chamado uma vez para cada aluno, mas sim **uma única vez** para resolver o campo disciplinas em **todos** os alunos que estão sendo processados na requisição.